SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Yuvaraj D (192421230) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

t

Yuvaraj D

RUBRICS FOR CALCULATION:

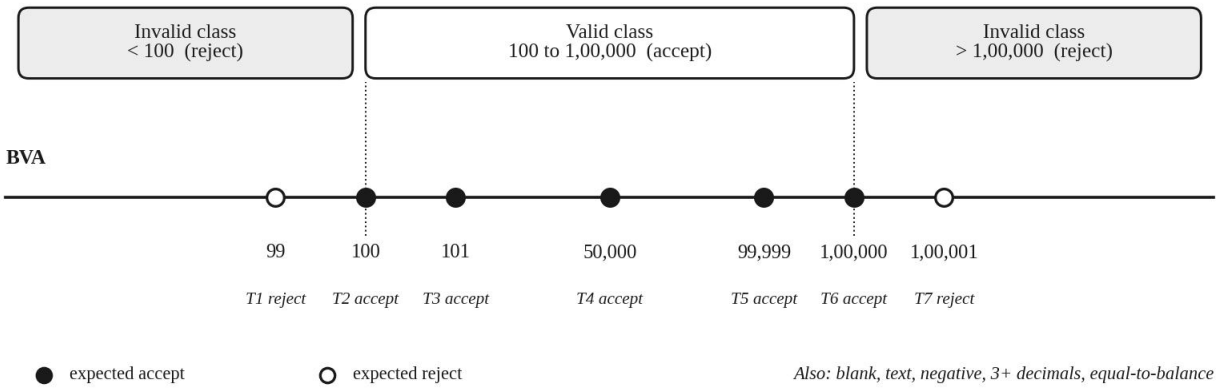
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Answers

Question 1: Fund Transfer Failures at Boundary Limits

Problem Understanding. The transfer module fails at the minimum and maximum limits, so the current suite checks typical amounts and misses the edges where off-by-one errors (< written instead of <=) occur. Random testing seldom lands on exact limits. Assumed limits: minimum Rs 100, maximum Rs 1,00,000.

Application of Concepts. Equivalence partitioning gives three classes: below minimum (invalid), 100 to 1,00,000 (valid) and above maximum (invalid), plus blank and non-numeric input. Boundary value analysis adds 99, 100, 101, 99,999, 1,00,000 and 1,00,001, with 50,000 as the nominal value (Figure 1).



Analysis and Decision. Random testing is cheap and can reveal unexpected inputs, but it is unrepeatable, gives no coverage measure and has a very small chance of choosing six exact values from lakhs of amounts. Decision: replace it as the main technique and keep only a small, seeded fuzz round as a supplement.

Solution Design. Use a layered design: (1) ECP for classes, (2) BVA for limits, (3) a decision table for balance, daily limit and account status, (4) automation in the regression pack, and (5) logged random seeds for the supplementary fuzz round.

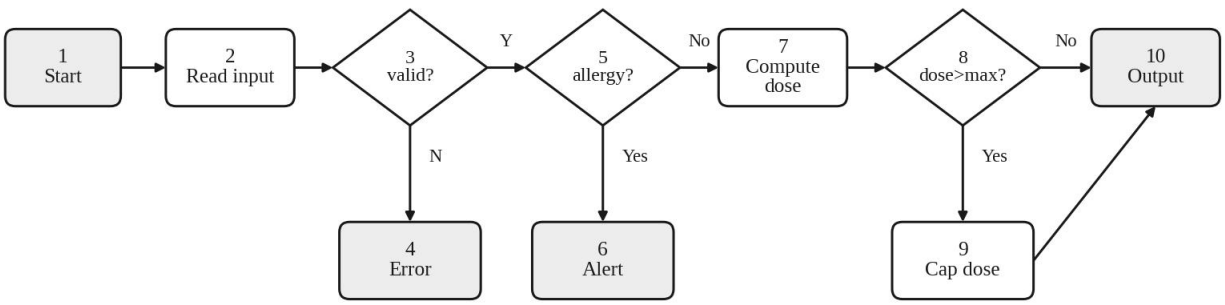
Justification. About ten focused cases replace thousands of random ones. They are repeatable, traceable to requirements and cover every partition and edge, giving the maximum defect detection per test.

Question 2: Post-Deployment Defects in a Healthcare Module

Problem Understanding. Critical defects escaped because tests followed the normal path only and no adequacy criterion was fixed. The module is conceptualised as a dose calculator that validates input, checks allergy, computes the dose and caps it at a safe maximum.

Application of Concepts. The control flow graph (Figure 2) has three decisions, so V(G) = 3 + 1 = 4. The independent paths are invalid input, allergy alert, capped dose and normal dose. The normal-dose path alone gives high statement coverage but leaves three paths unexecuted.

Conceptual CFG: dose-calculation module



Shaded nodes (1, 4, 6, 10) are entry/exit points; all exits join one final node.

V(G) = 3 decisions + 1 = 4 basis paths: P1 = 3(N); P2 = 3(Y),5(Yes); P3 = 3(Y),5(No),8(Yes); P4 = 3(Y),5(No),8(No)

Analysis and Decision. Statement coverage is the weakest criterion: the alert and cap statements run only when their decisions are true, so branch coverage stays incomplete exactly where patient safety depends on it. Branch coverage is the minimum here, and path coverage is feasible because the module has no loops.

Solution Design. Write one test per basis path (four tests), add boundary doses at max-1, max and max+1, and measure statement, branch and path coverage with a coverage tool. Any uncovered code must be tested or justified as dead.

Justification. A handful of tests gives 100% branch and path coverage and exercises every safety decision both ways. This lowers the risk of wrong doses, makes coverage measurable and gives auditable evidence for a regulated domain.

Question 3: Input Combinations in a Retail E-commerce System

Problem Understanding. Price, quantity and discount were tested one field at a time, so errors that appear only for particular combinations (for example a valid discount on a zero price, or two invalid fields hiding each other) were never exercised.

Application of Concepts. ECP gives valid and invalid classes per field: price > 0, quantity 1 to 100, discount 0 to 50%. BVA adds 0 and 0.01 for price; 0, 1, 100 and 101 for quantity; and -1, 0, 50 and 51 for discount. A decision table (Figure 3) combines the three conditions into 2^3 = 8 rules.

Decision table for Price (C1), Quantity (C2), Discount (C3) - 3 conditions give 2^3 = 8 rules

Condition / action	R1	R2	R3	R4	R5	R6	R7	R8
C1 Price > 0	Y	N	Y	Y	N	N	Y	N
C2 Qty 1 to 100	Y	Y	N	Y	N	Y	N	N
C3 Discount 0 to 50%	Y	Y	Y	N	Y	N	N	N
Expected action	Total	Err P	Err Q	Err D	Err P+Q	Err P+D	Err Q+D	Err all

BVA values added per field: Price 0, 0.01, 99999.99 | Qty 0, 1, 100, 101 | Discount -1, 0, 50, 51

Check: every invalid field must raise its own error even when other fields are also invalid (no error masking).

Analysis and Decision. Single-field testing finds field-level faults but cannot find interaction faults or error masking. ECP and BVA give coverage per field, not per combination, so the decision table supplies the missing combinations at a manageable size.

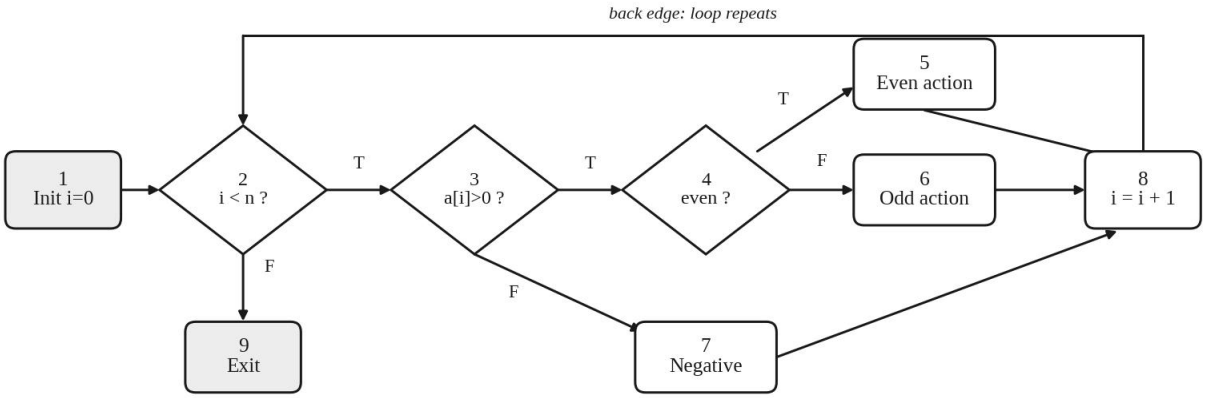
Solution Design. Execute one test per rule (eight tests) and take the Y and N data of every rule from the BVA values, so boundaries are checked inside combinations. Use the expected total P x Q x (1 - D) as the oracle for the valid rule.

Justification. Eight rules cover every combination of validity classes, and boundary data keep defect detection high. Completeness can be shown by listing each condition and rule as covered, without exhaustive enumeration.

Question 4: Nested Conditions and Loops with Partial Path Coverage

Problem Understanding. The module has a loop with nested conditions, and only part of the paths were covered. This indicates an incomplete control flow graph (back edge or compound conditions not modelled) or independent paths that were never listed.

Application of Concepts. The CFG in Figure 4 has three predicates (loop test, positive check, even check), so V(G) = 3 + 1 = 4, which agrees with E - N + 2 = 11 - 9 + 2. The basis paths are: skip the loop, negative element, even element and odd element.



V(G) = 3 + 1 = 4. P1: 1-2-9 P2: 1-2-3-7-8-2-9 P3: 1-2-3-4-5-8-2-9 P4: 1-2-3-4-6-8-2-9

Analysis and Decision. A missed independent path hides the code on it. The negative branch, the odd branch and the zero-iteration case are typical omissions, and faults there reach production. Each basis path adds at least one edge not covered earlier, so omitting one leaves edges untested.

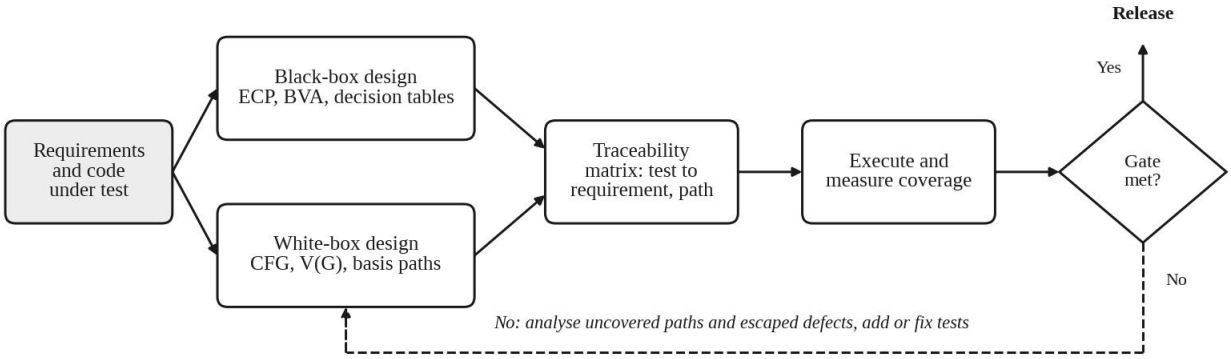
Solution Design. Create one test per basis path using data such as [ ], [-3], [4], [5] and a mixed array. Add loop tests for zero, one, two and maximum iterations. Accept the suite only when all basis paths and 100% branch coverage are reported.

Justification. Basis path testing is the smallest set that guarantees every edge and statement runs at least once. With loop tests it covers the main fault locations at modest cost and gives a measurable completion criterion.

Question 5: Production Defects Despite Black-box and White-box Testing

Problem Understanding. Both approaches were used, but separately: black-box cases were not linked to code paths and white-box cases were not linked to requirements. Coverage figures were reported but not used as a release criterion, so defects escaped.

Application of Concepts. ECP and BVA create requirement-based tests for each input, while the CFG, V(G) and basis paths create structure-based tests. A traceability matrix (Figure 5) joins the two, and coverage is measured on the combined suite.



Gate: 100% requirement coverage, 100% branch coverage, all basis paths executed, no open critical defect.

Analysis and Decision. Black-box testing finds wrong or missing behaviour but cannot show which code ran. White-box testing shows structure but cannot find omitted requirements. The decision is to treat them as complementary and to use coverage as a gate, not a report.

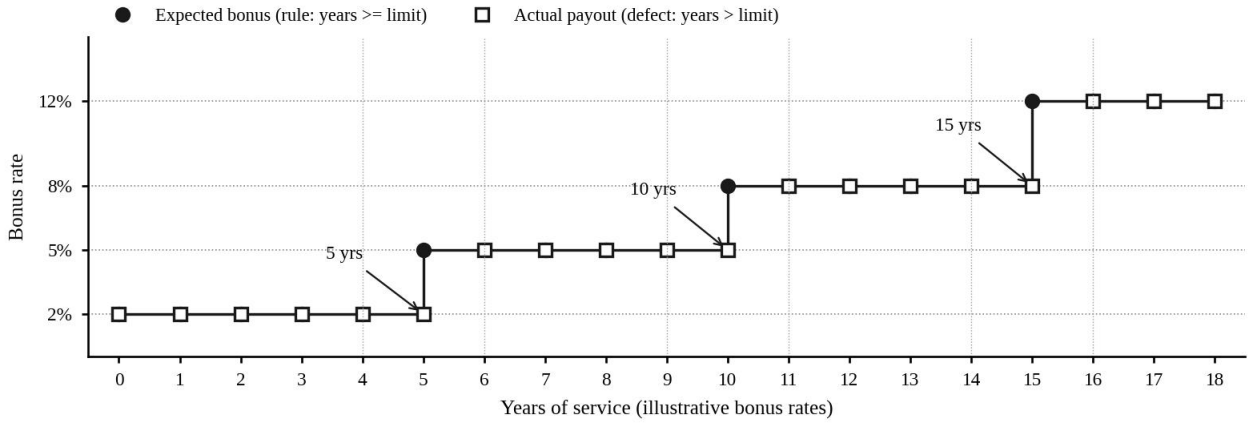
Solution Design. Phase 1: design ECP and BVA tests. Phase 2: draw CFGs and list basis paths for critical modules. Phase 3: map tests to requirements and paths, run them and measure statement, branch and path coverage. Phase 4: analyse gaps; every escaped defect adds a test and a root-cause note.

Justification. Requirement coverage guards against omitted behaviour and path coverage guards against untested logic. The gate and the feedback loop prevent defects instead of only detecting them, which is essential in a financial application.

Question 6: Incorrect Bonus Payouts at 5, 10 and 15 Years

Problem Understanding. The payout is wrong exactly at the tier thresholds. Five, ten and fifteen years are wrong while other values are right, which points to a relational operator fault: the code tests years > 5 where the rule is years >= 5. Earlier tests used values far from the thresholds.

Application of Concepts. ECP gives tiers of 0 to 4, 5 to 9, 10 to 14 and 15+ years, and rating classes low, average and high. BVA selects 4, 5, 6, 9, 10, 11, 14, 15 and 16 years (Figure 6), plus 0 years and an upper limit such as 40 years.



Analysis and Decision. Exploratory testing depends on the tester's intuition, is hard to repeat and cannot prove that every threshold was checked. Payroll rules are exact numeric thresholds, so a technique that targets the threshold itself is more reliable and auditable.

Solution Design. Build the suite from nine boundary tests with an average rating, three rating tests at one typical year per tier and two invalid inputs (negative years, missing rating). Record expected payouts in a table and automate the run for every payroll change.

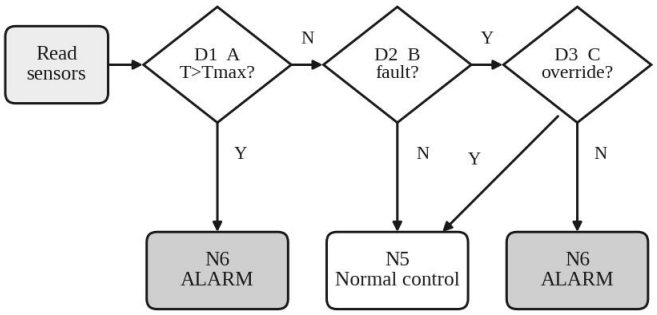
Justification. BVA and ECP are superior here because the defect sits on the boundary: the tests at 5, 10 and 15 years expose all three failures at once, they are repeatable and they give audit evidence. Exploratory sessions may still supplement them.

Question 7: Alarm Failure in a Temperature Control System

Problem Understanding. The alarm did not trigger in one sensor-malfunction sequence. The logic is conceptualised as D = A or (B and not C): A is over-temperature, B is sensor fault and C is maintenance override. The failure comes from untested operand combinations, not untested statements.

Application of Concepts. The CFG (Figure 7) has three decisions, giving V(G) = 4. Branch coverage needs only two tests, and short-circuit evaluation skips B and C whenever A is true, so the effect of the sensor-fault condition is never checked.

CFG: alarm logic D = A or (B and not C)



A: temperature above limit; B: sensor malfunction; C: maintenance override on.

Branch coverage needs only 2 tests; short-circuit hides B and C when A is true.

MC/DC set: n + 1 = 4 tests

Test	A	B	C	D	Pair proves
T1	0	1	0	1	B (T2), C (T3)
T2	0	0	0	0	B (T1), A (T4)
T3	0	1	1	0	C (T1)
T4	1	0	0	1	A (T2)

100% MC/DC also gives 100% branch coverage

Analysis and Decision. Branch coverage is inadequate for safety-critical logic because it never proves that each condition can change the decision on its own. MC/DC is therefore required, and it is feasible: n conditions need n + 1 tests, not 2^n.

Solution Design. Run the four tests T1 to T4 in the table, where each operand has a pair of tests in which only that operand changes and the result changes. Add malfunction sequences such as stuck value, out-of-range value and no signal.

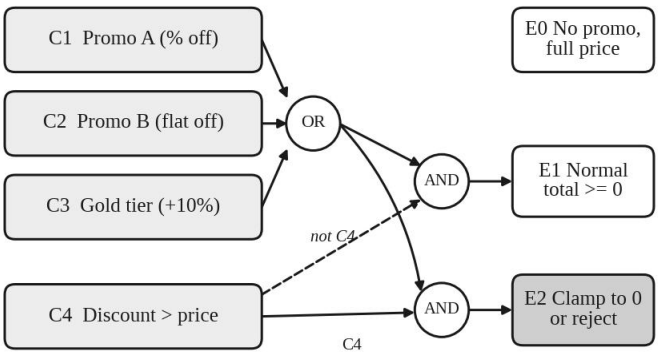
Justification. MC/DC finds masked-condition faults that branch coverage cannot, with only four tests, and is the criterion used for safety-critical software. Reliability rises because each fault condition is shown to raise the alarm independently.

Question 8: Negative Totals from Multiple Promo Codes

Problem Understanding. Promo codes were tested individually, so the combination of two codes and a tier discount, which can exceed the price, was never tried. Price, promo type and user tier interact and cannot be tested in isolation.

Application of Concepts. Cause-effect graphing (Figure 8) defines the causes: promo A (percentage), promo B (flat), gold tier and a derived cause C4, total discount greater than price. The effects are normal total, clamp to zero or reject, and full price. The graph converts into a decision table of eight rules.

Cause-effect graph



Constraint: C1 and C2 may combine (stacking); the OR node is the 'any discount' cause.

Reduced test set: 8 rules instead of 16

Rule	A	B	Gold	C4	Expected
R1	N	N	N	N	Full price
R2	Y	N	N	N	Normal total
R3	N	Y	N	N	Normal total
R4	Y	Y	N	N	Stack, total >= 0
R5	Y	Y	N	Y	Clamp to 0
R6	N	Y	Y	Y	Clamp to 0
R7	Y	Y	Y	Y	Clamp to 0
R8	N	N	Y	N	Normal total

Analysis and Decision. Isolated tests give single-promo results and never reach the over-discount cause, which exists only when causes combine. Exhaustive testing needs 16 combinations, and many are redundant because the effect depends only on whether any discount applies and whether it exceeds the price.

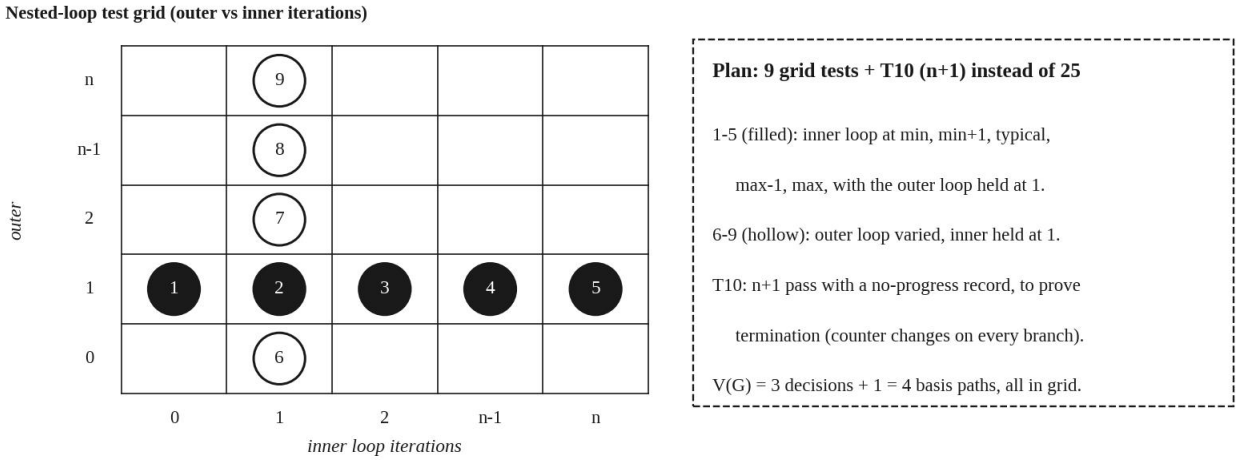
Solution Design. Execute rules R1 to R8, using boundary prices (0.01 and an amount equal to the discount) in R5 to R7 to verify the clamp. Assert total >= 0 in every test and add a stacking limit of one percentage code and one flat code.

Justification. The reduced set covers all effects and every cause combination that matters with half the tests, and the total >= 0 check makes each test detect the reported defect immediately.

Question 9: Infinite Loop Crashes in a Database Migration Tool

Problem Understanding. Statement coverage was high, yet infinite loops appeared in production. Statement coverage shows only that each line ran once; it says nothing about how loops end. A typical cause is a branch (such as skipping an invalid record) that returns to the loop without updating the counter.

Application of Concepts. Path testing targets independent paths and loop boundaries. For two nested loops with one record-validity decision, V(G) = 3 + 1 = 4. Loop tests follow the grid in Figure 9: inner loop at 0, 1, 2, n-1 and n iterations, then the outer loop varied, plus an n+1 termination probe.



Analysis and Decision. One record passing through the body executes every statement yet never takes the skip branch or the zero-iteration case. Statement coverage is therefore insufficient. Branch coverage and loop-boundary tests are the minimum, and V(G) fixes the minimum number of independent paths at four.

Solution Design. Create four basis path tests, nine loop-boundary tests from the grid and one no-progress test with invalid records only. Run all tests under a watchdog timeout in the harness to detect non-termination, and track branch and basis-path coverage.

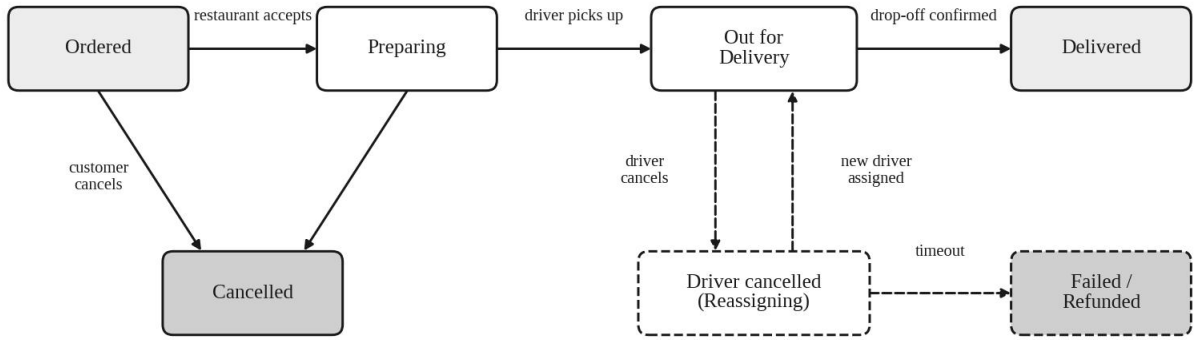
Justification. V(G) gives a measurable lower bound on tests, the loop-boundary tests expose termination faults, and the watchdog turns an infinite loop into a clear failing test before production.

Question 10: Real-time Tracker Stuck After Driver Cancellation

Problem Understanding. The tracker was designed with four states and no behaviour for a driver cancelling mid-delivery. No next state is defined, so users see the app stuck. The behaviour depends on the current state and the event, not on input values alone.

Application of Concepts. State transition testing models states, events and actions (Figure 10) and adds the missing states: Driver cancelled (Reassigning), Failed or Refunded, and Cancelled. Scenarios: S1 normal delivery; S2 cancel, reassign, delivered; S3 cancel, timeout, refund; S4 customer cancels; S5 invalid event.

State-transition model with the missing driver-cancel states



Dashed = state/transition absent from the current design and tests. Invalid events (e.g. 'driver cancels' while Preparing) must be ignored safely.

Analysis and Decision. ECP and BVA treat inputs in isolation and test one state at a time, whereas the stuck state appears only after a sequence of events. State transition testing exposes missing and invalid transitions through sequences, which is why it is chosen here.

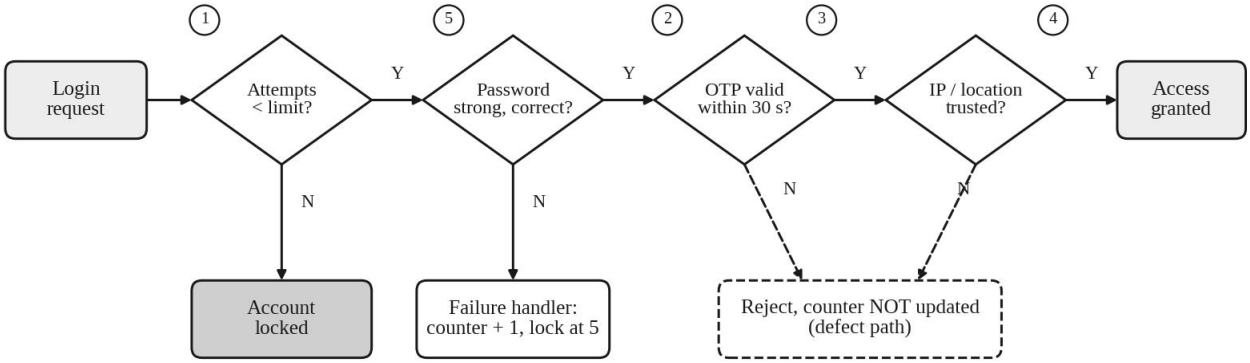
Solution Design. Build a state-event table. Cover every valid transition (0-switch), pairs of consecutive transitions around Out for Delivery (1-switch) and every invalid state-event pair. Add network loss and app restart during reassignment.

Justification. The technique attacks the logic gap directly: each transition and each undefined event gets a test, so no state is a dead end and the user always sees a defined state or a refund.

Question 11: Brute-Force Vulnerability in Multi-factor Authentication

Problem Understanding. The module combines password strength, OTP timing and IP location, yet brute-force attempts succeeded. Black-box tests checked valid and invalid credentials and white-box checks covered statements, but neither examined whether every failure path limits repeated attempts.

Application of Concepts. Formal path analysis builds the flow in Figure 11: five decisions, so V(G) = 6, and every failure path is traced to see whether the attempt counter is updated. Error guessing then lists likely attack inputs (probes 1 to 5) from experience of authentication failures.



Error-guessing probes: (1) rapid repeated tries (2) OTP at 29, 30, 31 s (3) OTP reuse (4) spoofed X-Forwarded-For (5) odd password characters

Path analysis: every failure path must reach the failure handler; the dashed path does not.

Analysis and Decision. Path analysis finds the structural fault: the OTP and IP failure paths bypass the failure handler, so the counter is never incremented. Error guessing finds the attack that exploits it, such as unlimited OTP tries. Neither technique alone is sufficient.

Solution Design. Hybrid plan: (1) derive basis paths, (2) attach an error-guess probe to each failure path, (3) add negative tests such as 1,000 rapid attempts and parallel sessions, (4) fix the code so every failure calls the handler, (5) re-run with coverage and a penetration scan.

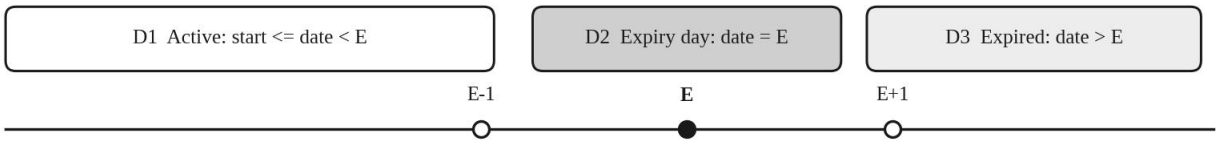
Justification. Error guessing brings attacker thinking and path analysis brings completeness. Together they show that every failure path is covered and lockout works, which measurably improves the security posture of the module.

Question 12: Upgrade on the Day of Expiry in an E-learning Platform

Problem Understanding. Errors occur when a user upgrades exactly on the expiry day. Existing tests treated dates as only valid or expired, so the expiry day, where the expiry job and the upgrade can conflict, was not a class of its own. Time zones add further risk.

Application of Concepts. ECP on dates (Figure 12): D1 active (start <= date < E), D2 expiry day (date = E) and D3 expired (date > E). ECP on subscriptions: Free, Basic and Premium. Actions: upgrade, renew and downgrade. Boundary dates E-1, E and E+1 combine with each action.

Equivalence classes on the expiry date E, and the upgrade test grid



Upgrade type	E-1	E (boundary)	E+1
Basic to Premium	Upgrade, days carried	Upgrade, no access gap	Treated as new purchase
Premium renewal	Extend from E	Extend, no lapse	Renew from today
Premium to Basic	Downgrade at E	Downgrade at 23:59 local	Already Free, no change

Extra checks: server UTC vs local IST midnight, leap day, daylight-saving regions, and a payment that completes after E.

Analysis and Decision. The earlier criteria, one active and one expired date, were inadequate because the correct behaviour at E differs from both neighbours. The decision is to treat E as a separate partition and test it for each tier transition.

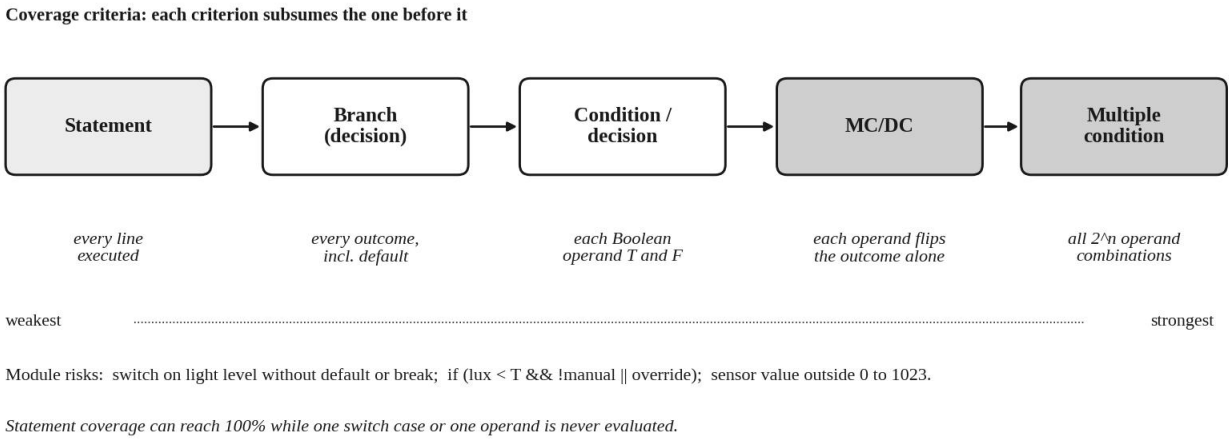
Solution Design. Execute the nine grid cases in Figure 12 (three transitions by three dates) plus four extra tests: 23:59 and 00:00 local time, a UTC and IST mismatch, a leap day, and a payment completing after E. Check course access after each transition.

Justification. The grid covers every date class with every transition type, so each combination that affects user access is exercised once. This gives complete, non-redundant coverage and transitions without access gaps or double charges.

Question 13: Smart-home Lighting Controller: Branch and Condition Coverage

Problem Understanding. The dimming algorithm contains switch-case statements and Boolean expressions over sensor readings. Relying on statement coverage lets a suite claim success while a case, a default branch or an operand has never been evaluated.

Application of Concepts. Branch coverage requires every case, including default, and each true and false outcome. Condition coverage requires every operand in an expression such as (lux < T && !manual) || override to take both values. Figure 13 shows how the criteria build on each other.



Analysis and Decision. Risks of statement coverage: a missing default or break is invisible, short-circuit evaluation hides operands, and out-of-range sensor values never reach tests. Branch plus condition coverage, with MC/DC for the critical expression, is the chosen minimum.

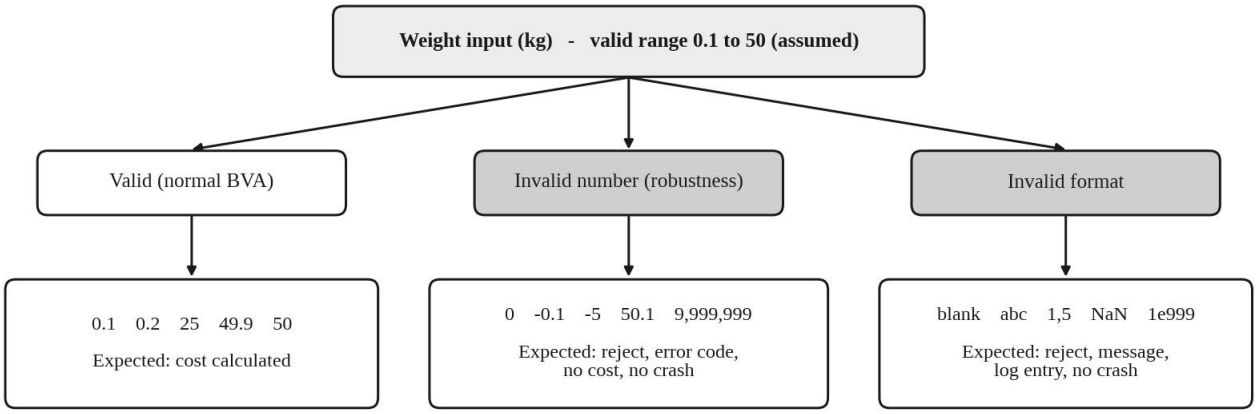
Solution Design. Derive a test for each case label, add out-of-range and boundary lux values at each threshold, and add four or five MC/DC-style tests for the compound expression. Add simulator or hardware-in-the-loop tests with noisy, stuck and missing sensor values.

Justification. These tests expose missing logical paths before integration, so unexpected sensor values lead to defined, fail-safe dimming behaviour, which reduces hardware-software integration failures.

Question 14: Zero and Negative Weights in Shipping-cost Calculation

Problem Understanding. The weight field was checked only with valid weights, so zero and negative values passed validation and produced meaningless costs. The input validation tests did not include invalid boundaries.

Application of Concepts. Robustness BVA extends normal BVA with values just outside the range. For the assumed range 0.1 to 50 kg (Figure 14): 0, -0.1 and -5 below the minimum; 0.1, 0.2, 25, 49.9 and 50 inside; 50.1 and 9,999,999 above; and malformed values such as blank, text and NaN.



Negative scenarios: min-1, zero, negative, over-max, extreme and malformed values.

Analysis and Decision. Without negative tests, system behaviour for bad data is undefined: the cost could be zero or negative and cause a billing error. The analysis shows that validation of weight must run before the cost formula that uses zone and speed.

Solution Design. Combine the tree leaves with representative zone and delivery-speed pairs. Assert that invalid input is rejected with a clear message, no cost is computed, nothing is written to the order and no exception escapes.

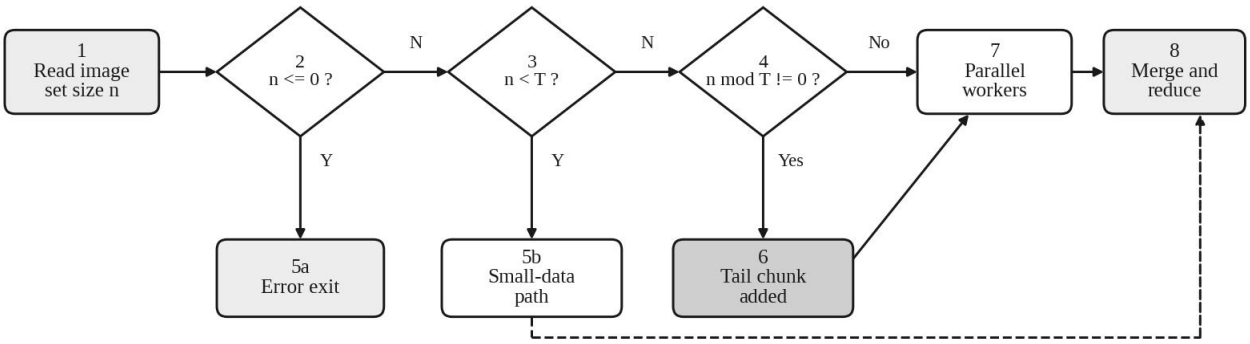
Justification. Real users and integrations do send bad data. Negative scenarios show that the system fails safely instead of charging wrong amounts or crashing, which is what makes it robust.

Question 15: Inconsistent Results for Prime Data Sizes in Medical Imaging

Problem Understanding. Results differ only when the data size n is prime. A prime n larger than the thread count T is never divisible by T, so the remainder (tail chunk) branch always runs, and this is the probable fault location. Random sizes rarely target it systematically.

Application of Concepts. Basis path testing on the partitioning module (Figure 15) has three decisions, so V(G) = 4: invalid size, n < T, evenly divisible and remainder. Primes (2, 3, 5, 7, 11, 13, 101) exercise the remainder path and multiples of T exercise the divisible path.

CFG of the partitioning module (T = number of worker threads)



V(G) = 3 decisions + 1 = 4. P1: 1-2-5a P2: 1-2-3-5b-8 P3: 1-2-3-4-7-8 P4: 1-2-3-4-6-7-8

A prime n (greater than T) is never divisible by T, so it always takes the tail-chunk branch (P4).

Analysis and Decision. Random testing cannot guarantee that the remainder branch, the n < T case (chunk size zero) or the floating-point reduction order is exercised, and it has no oracle. The decision is structured basis path testing with a serial reference result as the oracle.

Solution Design. Execute paths P1 to P4 and compare the parallel output with the single-thread reference using a defined tolerance. Repeat each run with different T to expose race conditions, which basis paths alone do not cover, and add stress tests with large sizes.

Justification. Basis paths guarantee that every partitioning branch runs, the reference comparison proves numerical accuracy, and repeated runs show stability. The prime-size defect becomes reproducible and deterministic.